Listas
Lineares Dinâmicas

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Revisão

- Uma das formas mais simples de **interligar os elementos** de um conjunto é por meio de uma **lista**.

- Listas são **estruturas flexíveis**, que podem **crescer** ou **diminuir** de tamanho **durante a execução** de um programa.

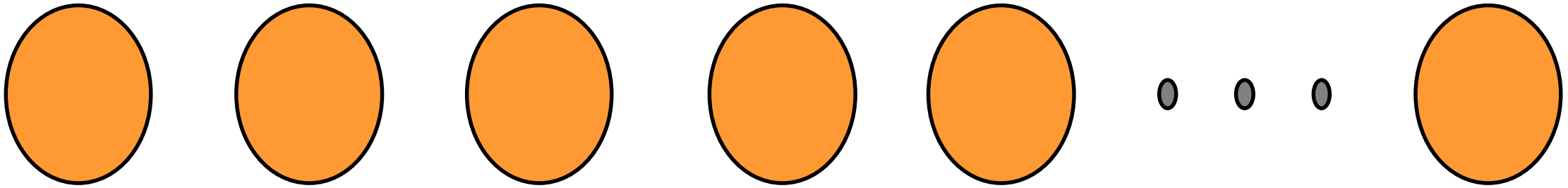
Lista Linear

- Uma **lista linear** é uma sequência de zero ou mais itens $x_1, x_2, x_3, ..., x_n$ na qual $\mathbf{x_i}$ é de determinado tipo e **n** representa o tamanho da lista.

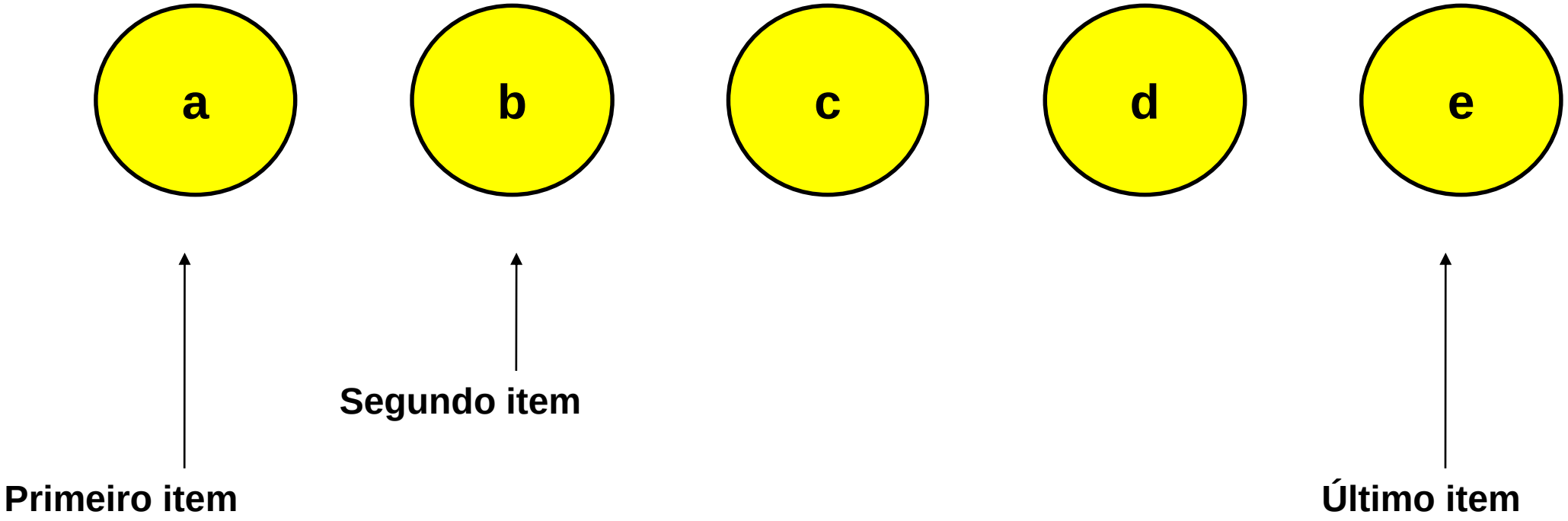
- Assumindo que $n \geq 1$, tem-se que x_1 é o primeiro item da lista e x_n é o último item.

- Em geral, x_i precede x_{i+1} e sucede x_{i-1}.

Exemplo



Exemplo



Lista Linear

- Para criar um **tipo abstrato de dados** Lista, é necessário definir um **conjunto de operações** sobre as variáveis do tipo Lista.

- O conjunto de operações a ser definido **depende de cada aplicação**, não existindo um conjunto **adequado para todas as aplicações**.

Implementação
Listas Lineares

- Existem **diferentes estruturas** que podem ser utilizadas na **implementação** de listas lineares.

- Cada estrutura apresenta **vantagens** e **desvantagens** particulares.

Implementação
Listas Lineares

- **Implementações mais utilizadas**
  - Arranjos (vetores)
  - Apontadores (ponteiros)

Implementação
Listas Lineares

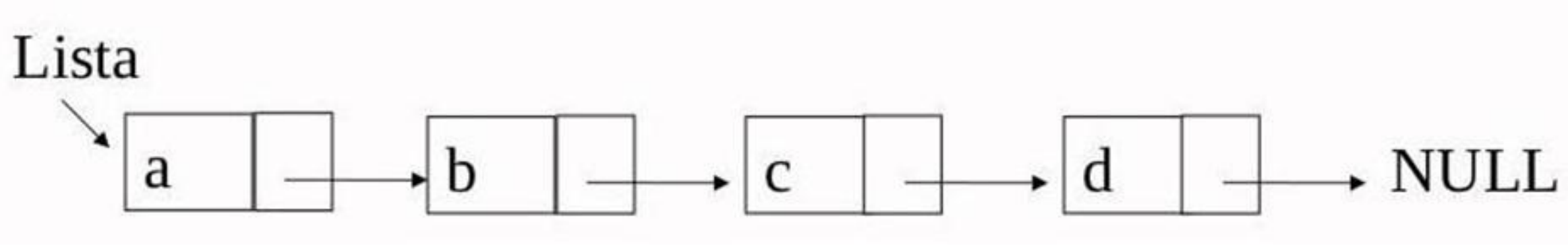
- **Implementações mais utilizadas**
  - Arranjos (vetores)
  - **Apontadores** (**ponteiros**)

Implementação por Apontadores

- Na implementação de listas por apontadores, cada item da lista é **encadeado** com o seguinte mediante um **variável** do tipo **apontador**, ou **ponteiro**.

- Este **tipo de implementação** possibilita utilizar **posições não sequenciais** de memória.

Representação



Operações
Listas Lineares

- As seguintes operações podem ser utilizadas em diferentes aplicações:
  - **inicializar** a lista
  - **verificar** se a lista está **vazia**
  - **inserir item** na lista
  - **retirar item** da lista
  - **imprimir os itens** da lista

Implementação por Apontadores

```
#include <stdlib.h>
#include <stdio.h>
```

```
typedef struct TipoItem *TipoApontador;

typedef struct TipoItem {
  int conteudo;
  TipoApontador proximo;
} TipoItem;
```

Implementação por Apontadores

```
typedef struct {
  TipoApontador primeiro, ultimo;
} TipoLista;
```

```
void Inicializa(TipoLista *lista){
  lista->primeiro =
       (TipoApontador) malloc (sizeof(TipoItem));
  lista->primeiro->proximo = NULL;
  lista->ultimo = lista->primeiro;
}
```

Implementação por Apontadores

```
int Vazia(TipoLista lista){
  return (lista.primeiro == lista.ultimo);
}
```

```
void Insere(int elemento, TipoLista *lista){
  lista->ultimo->proximo =
       (TipoApontador) malloc (sizeof(TipoItem));
  lista->ultimo = lista->ultimo->proximo;
  lista->ultimo->conteudo = elemento;
  lista->ultimo->proximo = NULL;
}
```

Implementação por Apontadores

```
void Retira(TipoApontador ponteiro,
                                TipoLista *lista, int elemento){
  TipoApontador p_auxiliar;
  if (Vazia(*lista) ||
                ponteiro == NULL || ponteiro->proximo == NULL) {
    printf(" Erro! Lista vazia ou posicao nao existe \n");
    return;
  }
  p_auxiliar = ponteiro->proximo;
  elemento = p_auxiliar->conteudo;
  ponteiro->proximo = p_auxiliar->proximo;

  if (ponteiro->proximo == NULL)
    lista->ultimo = ponteiro;
  free(p_auxiliar);
}
```

Implementação por Apontadores

```
void Imprime(TipoLista lista){
  TipoApontador p_auxiliar;
  p_auxiliar = lista.primeiro->proximo;

  while (p_auxiliar != NULL) {
    printf("%d\n", p_auxiliar->conteudo);
    p_auxiliar = p_auxiliar->proximo;
  }
}
```

Observações

- A **inserção de um novo item** após o **último item** é realizada de maneira simples.

- Já a inserção de um novo item no **meio da lista** requer uma ou mais **modificações nos ponteiros**.

Observações

- Outras operações poderiam ser consideradas, além das apresentadas.

- **Exemplos**
  - **Pesquisar** um item da lista
  - **Contar** o número de itens da lista

Reforço

- O **conjunto de operações** a ser definido **depende de cada aplicação**.

Observações

- A implementação de listas por meio de apontadores permite **inserir** ou **retirar itens** do meio da lista com uma **certa facilidade**.

- Esse aspecto é importante quando a lista **tem de ser mantida em ordem**.

Observações

- Para aplicações em que **não existe nenhuma previsão** sobre o **tamanho da lista**, é conveniente utilizar listas lineares encadeadas por apontadores.

- Nesse caso o tamanho máximo da lista **não precisa** ser definido de forma **antecipada**.

Implementação por Arranjos

- **Vantagens**
  - **Compartilhamento de memória**, pois os itens podem fazer parte de **mais de uma estrutura**.
  - **Facilidade** para **inserção** e **remoção de itens**. Nesse caso, é preciso **modificar** um ou mais **ponteiros** e **nenhum deslocamento** é necessário.

Implementação por Arranjos

- **Desvantagens**
  - **Transferência de dados**, uma vez que os dados estão **espalhados pela memória**.
  - **Acesso**, que deve ser realizado de forma **serial**. A princípio, **não é possível** acessar **qualquer item** a **qualquer momento**.

Referências

- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.
- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.
- Estruturas de Dados, Nina Edelweiss e Renata Galante, 2008.
- Aulas Professor Rodolfo Miranda Pereira, UEM.